

Designing and Implementing a Banking System with PL/SQL

This document describes the construction of a small banking system for Gagnef Savings Bank. It walks through the process step by step, from designing the underlying data model to implementing the tables, constraints, triggers, functions, and procedures that make up the working system. The specification below outlines the tables required, along with their columns and data types. Together they form the foundation on which the rest of the system is built.

```
SQL> desc customer
```

Name	Null?	Type
-----	-----	-----
CUST_ID	NOT NULL	VARCHAR2(11)
FIRST_NAME	NOT NULL	VARCHAR2(25)
LAST_NAME	NOT NULL	VARCHAR2(25)
PASSWD	NOT NULL	VARCHAR2(6)

```
SQL> desc account_type
```

Name	Null?	Type
-----	-----	-----
ACCTY_ID	NOT NULL	NUMBER(6)
ACCTY_NAME	NOT NULL	VARCHAR2(20)
PRESENT_INTEREST	NOT NULL	NUMBER(5,2)

```
SQL> desc interest_change
```

Name	Null?	Type
-----	-----	-----
INTCH_ID	NOT NULL	NUMBER(6)
ACCTY_ID	NOT NULL	NUMBER(6)
INTEREST	NOT NULL	NUMBER(5,2)
DATE_TIME	NOT NULL	DATE

```
SQL> desc account
```

Name	Null?	Type
-----	-----	-----
ACC_ID	NOT NULL	NUMBER(8)
ACCTY_ID	NOT NULL	NUMBER(6)
DATE_TIME	NOT NULL	DATE
BALANCE	NOT NULL	NUMBER(10,2)

```
SQL> desc account_owner
```

Name	Null?	Type
-----	-----	-----
ACCOW_ID	NOT NULL	NUMBER(9)
CUST_ID	NOT NULL	VARCHAR2(11)
ACC_ID	NOT NULL	NUMBER(8)

```
SQL> desc withdrawal
```

Name	Null?	Type
-----	-----	-----
WIT_ID	NOT NULL	NUMBER(9)
CUST_ID	NOT NULL	VARCHAR2(11)
ACC_ID	NOT NULL	NUMBER(8)
AMOUNT	NOT NULL	NUMBER(10,2)
DATE_TIME	NOT NULL	DATE

```
SQL> desc deposition
```

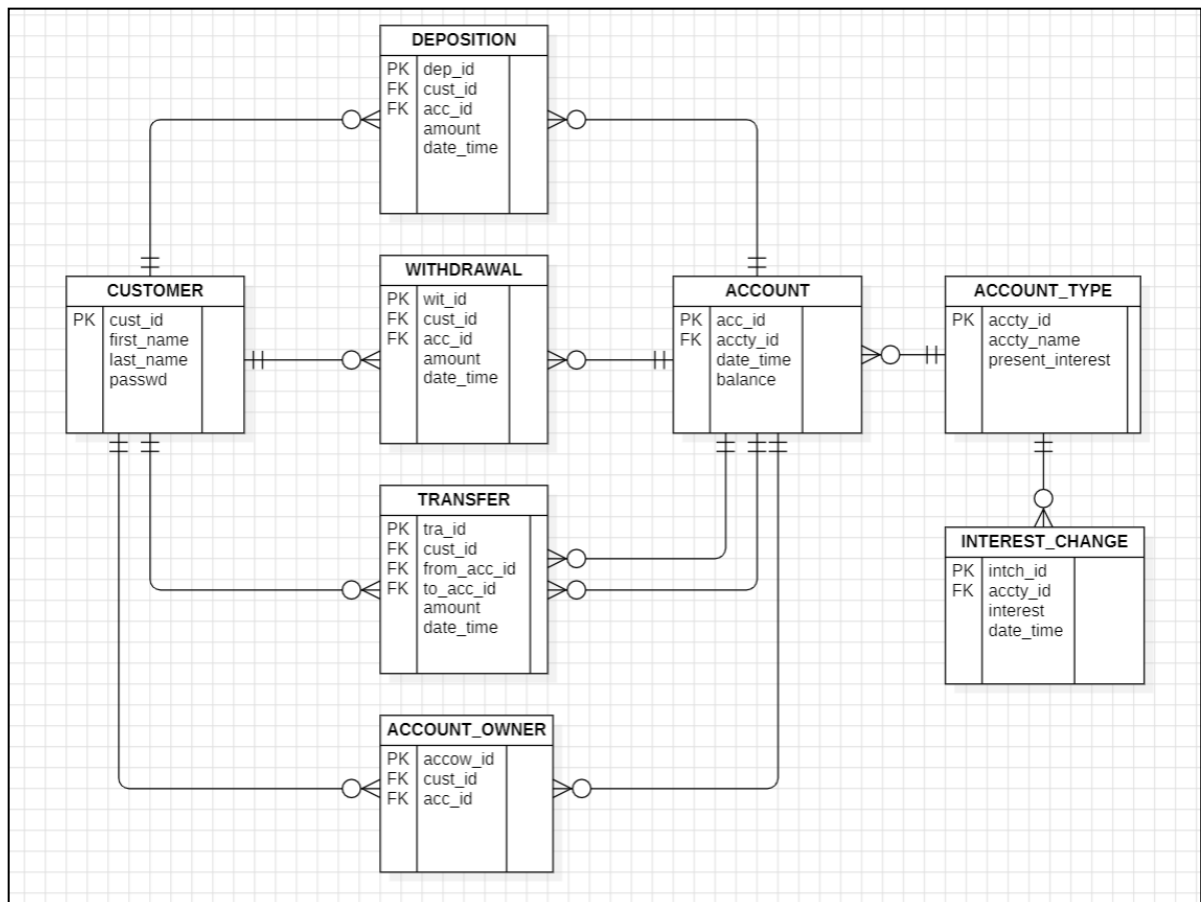
Name	Null?	Type
DEP_ID	NOT NULL	NUMBER(9)
CUST_ID	NOT NULL	VARCHAR2(11)
ACC_ID	NOT NULL	NUMBER(8)
AMOUNT	NOT NULL	NUMBER(10,2)
DATE_TIME	NOT NULL	DATE

```
SQL> desc transfer
```

Name	Null?	Type
TRA_ID	NOT NULL	NUMBER(9)
CUST_ID	NOT NULL	VARCHAR2(11)
FROM_ACC_ID	NOT NULL	NUMBER(8)
TO_ACC_ID	NOT NULL	NUMBER(8)
AMOUNT	NOT NULL	NUMBER(10,2)
DATE_TIME	NOT NULL	DATE

Building the Data Model

We begin by mapping out the structure of the banking system. Each table's first column serves as its primary key, and referential integrity is established through consistent column naming. The one exception is the transfer table, where both `from_acc_id` and `to_acc_id` reference `account(acc_id)`. The system follows an *action-modeling approach*, meaning the operations of **withdrawing**, **depositing**, and **transferring** money are each represented by their own dedicated tables with descriptive names. Note that the `cust_id` column in the `customer` table holds a *person identification number*.



Implementing the Tables

With the data model defined, we now translate it into the database using SQL DDL. The following statements create each table and establish the primary and foreign key constraints that enforce the relationships between them.

```

/***** CUSTOMER TABLE *****/
create table customer(
cust_id varchar2(11) not null,
first_name varchar2(25) not null,
last_name varchar2(25) not null,
passwd varchar2(6) not null
);

alter table customer
add constraint customer_custid_pk primary key(cust_id);

/***** ACCOUNT_TYPE TABLE *****/
create table account_type(
accty_id number(6) not null,
accty_name varchar2(20) not null,
present_interest number(5,2) not null
);

alter table account_type
add constraint accounttype_acctyid_pk primary key(accty_id);

/***** INTEREST_CHANGE TABLE *****/
create table interest_change(
intch_id number(6) not null,
accty_id number(6) not null,
interest number(5,2) not null,
date_time date not null
);

alter table interest_change
add constraint interestchange_intchid_pk primary key(intch_id)
add constraint interestchange_acctyid_fk foreign key(accty_id) references
account_type(accty_id);

/***** ACCOUNT TABLE *****/
create table account(
acc_id number(8) not null,
accty_id number(6) not null,
date_time date not null,
balance number(10,2) not null
);

alter table account
add constraint account_accid_pk primary key(acc_id)
add constraint account_acctyid_fk foreign key(accty_id) references
account_type(accty_id);

```

```

/***** ACCOUNT_OWNER TABLE *****/
create table account_owner(
accow_id number(9) not null,
cust_id varchar2(11) not null,
acc_id number(8) not null
);

alter table account_owner
add constraint accountowner_accowid_pk primary key(accow_id)
add constraint accountowner_custid_fk foreign key(cust_id) references
customer(cust_id)
add constraint accountowner_accid_fk foreign key(acc_id) references
account(acc_id);

/***** DEPOSITION TABLE *****/
create table deposition(
dep_id number(9) not null,
cust_id varchar2(11) not null,
acc_id number(8) not null,
amount number(10,2) not null,
date_time date not null);

alter table deposition
add constraint deposition_dep_id_pk primary key(dep_id)
add constraint deposition_custid_fk foreign key(cust_id) references
customer(cust_id)
add constraint deposition_accid_fk foreign key(acc_id) references
account(acc_id);

/***** WITHDRAWAL TABLE *****/
create table withdrawal(
wit_id number(9) not null,
cust_id varchar2(11) not null,
acc_id number(8) not null,
amount number(10,2) not null,
date_time date not null
);

alter table withdrawal
add constraint withdrawal_witid_pk primary key(wit_id)
add constraint withdrawal_custid_fk foreign key(cust_id) references
customer(cust_id)
add constraint withdrawal_accid_fk foreign key(acc_id) references
account(acc_id);

/***** TRANSFER TABLE *****/
create table transfer(
tra_id number(9) not null,
cust_id varchar2(11) not null,
from_acc_id number(8) not null,
to_acc_id number(8) not null,
amount number(10,2) not null,
date_time date not null
);

```

```

alter table transfer
add constraint transfer_traid_pk primary key(tra_id)
add constraint transfer_custid_fk foreign key(cust_id) references
customer(cust_id)
add constraint transfer_fromaccid_fk foreign key(from_acc_id) references
account(acc_id)
add constraint transfer_toaccid_fk foreign key(to_acc_id) references
account(acc_id);

```

Enforcing Password Rules

To protect data integrity, we add a trigger named `biufer_customer` that fires **before** any **insert** or **update** to the **passwd** column of the **customer** table. This trigger guarantees that every password is exactly six characters long. If the requirement isn't met, the transaction is halted and an error is reported.

```

create or replace trigger biufer_customer
before insert or update
of passwd
on customer
for each row
when (length(new.passwd) <> 6)
begin
    raise_application_error(-20001,'Password must contain 6 characters!');
end;
/

```

Adding New Customers

Next, we create a procedure called `do_new_customer` to streamline the process of registering new customers. It accepts four input parameters, **in order**: `cust_id`, `first_name`, `last_name`, `passwd`.

```

create or replace procedure do_new_customer(
p_cust_id    in customer.cust_id%type,
p_first_name in customer.first_name%type,
p_last_name  in customer.last_name%type,
p_passwd     in customer.passwd%type)
as
begin
    insert into customer(cust_id, first_name, last_name, passwd)
    values(p_cust_id, p_first_name, p_last_name, p_passwd);
    commit;
end;
/

```

Populating the Customer Table

We now add our first four customers using the `do_new_customer` procedure. Before doing so, we verify that the `biufer_customer` trigger behaves correctly by attempting to insert a password of the wrong length. The trigger test below should be run first, followed by the block that adds valid customers.

Trigger test:

```

begin
do_new_customer('861124-4478','Raul','Ortiz','qwe');
end;

```

The trigger test produces the following result:

```
ORA-20001: Password must contain 6 characters! ORA-06512: at
"SQL_KLXDRLKTEHZPQVKDMAISAPBIY.BIUFER_CUSTOMER", line 2 ORA-06512: at
"SQL_KLXDRLKTEHZPQVKDMAISAPBIY.DO_NEW_CUSTOMER", line 8 ORA-06512: at line 2
ORA-06512: at "SYS.DBMS_SQL", line 172
```

We then add the four customers:

```
-----Start copy and paste-----
BEGIN
do_new_customer('650707-1111','Tito','Ortiz','qwerTY');
do_new_customer('560126-1148','Margreth','Andersson','olle85');
do_new_customer('840317-1457','Mary','Smith','asdfgh');
do_new_customer('861124-4478','Vincent','Ortiz','qwel23');
COMMIT;
END;
/
-----End copy and paste-----
```

Generating Primary Keys Automatically

To simplify the assignment of primary key values, we create a sequence called `pk_seq`. This sequence supplies key values for the primary key columns of the transfer, deposition, withdrawal, account_owner, and interest_change tables, each wired up through its own trigger.

```
create sequence pk_seq;

create or replace trigger bifer_accountowner_pk
before insert
on account_owner
for each row
begin
    select pk_seq.nextval
    into :new.accow_id
    from dual;
end;
/

create or replace trigger bifer_withdrawal_pk
before insert
on withdrawal
for each row
begin
    select pk_seq.nextval
    into :new.wit_id
    from dual;
end;
/

create or replace trigger bifer_deposition_pk
before insert
on deposition
for each row
begin
    select pk_seq.nextval
    into :new.dep_id
    from dual;
end;
/
```

```

create or replace trigger bifer_transfer_pk
before insert
on transfer
for each row
begin
    select pk_seq.nextval
    into :new.tra_id
    from dual;
end;
/

create or replace trigger bifer_interestchange_pk
before insert
on interest_change
for each row
begin
    select pk_seq.nextval
    into :new.intch_id
    from dual;
end;
/

```

With the key generation in place, we can now load initial data into the `account_type`, `account` and `account_owner` tables.

```

-----Start copy and paste-----
INSERT INTO account_type(accty_id,accty_name,present_interest)
VALUES(1,'farmer account',2.4);
INSERT INTO account_type (accty_id,accty_name,present_interest)
VALUES(2,'potato account',3.4);
INSERT INTO account_type (accty_id,accty_name,present_interest)
VALUES(3,'hog account',4.4);
COMMIT;
INSERT INTO account(acc_id,accty_id,date_time,balance)
VALUES(123,1,SYSDATE - 321,0);
INSERT INTO account(acc_id,accty_id,date_time,balance)
VALUES(5899,2,SYSDATE - 2546,0);
INSERT INTO account(acc_id,accty_id,date_time,balance)
VALUES(5587,3,SYSDATE - 10,0);
INSERT INTO account(acc_id,accty_id,date_time,balance)
VALUES(8896,1,SYSDATE - 45,0);
COMMIT;
INSERT INTO account_owner(accow_id,cust_id,acc_id)
VALUES(pk_seq.NEXTVAL,'650707-1111',123);
INSERT INTO account_owner(accow_id,cust_id,acc_id)
VALUES(pk_seq.NEXTVAL,'560126-1148',123);
INSERT INTO account_owner(accow_id,cust_id,acc_id)
VALUES(pk_seq.NEXTVAL,'650707-1111',5899);
INSERT INTO account_owner(accow_id,cust_id,acc_id)
VALUES(pk_seq.NEXTVAL,'861124-4478',8896);
COMMIT;
-----End copy and paste-----

```

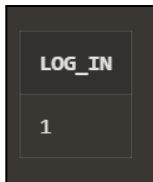
Handling Customer Logins

We create a function named `log_in` to manage authentication. It returns `1` if the login succeeds and `0` if it fails. A customer must supply two parameters to log in: their `cust_id` and `passwd`.

```
create or replace function log_in(
p_cust_id in customer.cust_id%type,
p_passwd in customer.passwd%type)
return number
as
v_result customer.cust_id%type;
begin
    select cust_id
    into v_result
    from customer
    where cust_id = p_cust_id
    and passwd = p_passwd;
    return 1;
exception
    when no_data_found then
    return 0;
end;
/
```

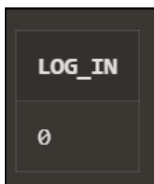
We confirm the function works by testing it with valid and invalid credentials:

```
select log_in('650707-1111','qwerTY') as log_in
from dual;
```



LOG_IN
1

```
select log_in('650707-1111','olle85') as log_in
from dual;
```



LOG_IN
0

Checking Account Balances

Next, we build a function called `get_balance` that returns the current balance of an account when given its account number (`acc_id`).

```
create or replace function get_balance(
p_acc_id in account.acc_id%type)
return account.balance%type
as
v_balance account.balance%type;
begin
    select balance
    into v_balance
    from account
    where acc_id = p_acc_id;
    return v_balance;
end;
/
```

The function test returns:

```
select get_balance(123) as current_balance
from dual;
```

CURRENT_BALANCE
0

Verifying Account Authority

We create a function called `get_authority` that takes two parameters from the `account_owner` table: `cust_id` and `acc_id`, and returns `1` if the customer is authorized to make withdrawals from the account, or `0` if they are not. This function will also be used during transfers to confirm that a customer has authority over the account identified by *from_acc_id*.

```
create or replace function get_authority(
p_cust_id in account_owner.cust_id%type,
p_acc_id in account_owner.acc_id%type)
return number
as
v_result number(1);
begin
    select count(accow_id)
    into v_result
    from account_owner
    where cust_id = p_cust_id
    and acc_id = p_acc_id;
    return v_result;
end;
/
```

The function test returns:

```
select get_authority('650707-1111',123) as authorization
from dual;
```

```
select get_authority('650707-1111',5899) as authorization
from dual;
```

AUTHORIZATION
1

```
select get_authority('650707-1111',8896) as authorization
from dual;
```

AUTHORIZATION
0

Updating Balances After a Deposit

We add a trigger called `aifer_deposition` that fires **after** an **insert** on the deposition table, ensuring the account balance is correctly updated whenever money is deposited.

```
create or replace trigger aifer_deposition
after insert
on deposition
for each row
begin
    update account
    set balance = balance + :new.amount
    where acc_id = :new.acc_id;
end;
/
```

Preventing Overdrafts on Withdrawals

To safeguard against overdrawing, we create a trigger called `bifer_withdrawal` that fires **before** an **insert** on the withdrawal table. It uses the `get_balance` function to confirm that a withdrawal never exceeds the available balance.

```
create or replace trigger bifer_withdrawal
before insert
on withdrawal
for each row
begin
    if ( :new.amount > get_balance(:new.acc_id) ) then
        raise_application_error(-20001,'You do not have enough money in your
account!');
    end if;
end;
/
```

Updating Balances After a Withdrawal

We add a trigger called `aifer_withdrawal` that fires **after** an **insert** on the withdrawal table, ensuring the balance is accurate once a withdrawal has been made.

```

create or replace trigger aifer_withdrawal
after insert
on withdrawal
for each row
begin
    update account
    set balance = balance - :new.amount
    where acc_id = :new.acc_id;
end;
/

```

Preventing Overdrafts on Transfers

Similarly, we create a trigger named `bifer_transfer` that fires **before** an **insert** on the transfer table, guaranteeing that a customer cannot move more money out of an account than it currently holds.

```

create or replace trigger bifer_transfer
before insert
on transfer
for each row
begin
    if ( :new.amount > get_balance(:new.from_acc_id) ) then
        raise_application_error(-20001, 'You do not have enough money in your
account!');
    end if;
end;
/

```

Updating Balances After a Transfer

The final trigger, `aifer_transfer`, fires **after** an **insert** on the transfer table. It keeps both accounts accurate by deducting the amount from the source account and adding it to the destination account.

```

create or replace trigger aifer_transfer
after insert
on transfer
for each row
begin
    update account
    set balance = balance - :new.amount
    where acc_id = :new.from_acc_id;

    update account
    set balance = balance + :new.amount
    where acc_id = :new.to_acc_id;
end;
/

```

Processing Deposits

We create a procedure called `do_deposition` that inserts a row into the **deposition** table. Once the transaction commits, it prints a message showing the account's updated balance.

```
create or replace procedure do_deposition(
p_cust_id in deposition.cust_id%type,
p_acc_id  in deposition.acc_id%type,
p_amount  in deposition.amount%type)
as
begin
    insert into deposition(cust_id, acc_id, amount, date_time)
    values(p_cust_id, p_acc_id,p_amount,sysdate) ;
    commit;
    dbms_output.put_line('The current balance of the account
'||p_acc_id||' is '||get_balance(p_acc_id)||' sek.');
```

Testing the Deposit Procedure

We verify that `do_deposition` works as intended and confirm the balance is updated correctly:

```
begin
    do_deposition('650707-1111',123,3240);
end;
/
```

```
Statement processed.
The current balance of the account 123 is 3240 sek.
```

```
select balance
from account
where acc_id = 123;
```

BALANCE
3240

Processing Withdrawals

We create a procedure called `do_withdrawal` that inserts a row into the **withdrawal** table. It declares an exception named `unauthorized`, which is raised via the `get_authority` function whenever a customer lacks authority over the account. If the customer is unauthorized, the transaction is stopped and the message *"Unauthorized user!"* is printed. If authorized, the procedure prints the account balance after the transaction.

```
create or replace procedure do_withdrawal (
p_cust_id in withdrawal.cust_id%type,
p_acc_id  in withdrawal.acc_id%type,
p_amount  in withdrawal.amount%type)
as
begin
  if (get_authority(p_cust_id, p_acc_id)=0) then
    raise_application_error(-20001,'Unauthorized user!');
  else
    insert into withdrawal(cust_id, acc_id, amount, date_time)
    values(p_cust_id, p_acc_id,p_amount ,sysdate) ;
    commit;
    dbms_output.put_line('The current balance of the account
'||p_acc_id||' is '||get_balance(p_acc_id)||' sek. ');
  end if;
end;
/
```

Testing the Withdrawal Procedure

We now test `do_withdrawal`, which also lets us confirm that the supporting triggers work. We check that a withdrawal larger than the available balance is rejected and that the balance updates correctly:

```
begin
  do_withdrawal('650707-1111',123,1000);
end;
/
```

```
Statement processed.
The current balance of the account 123 is 2240 sek.
```

```
select balance
from account
where acc_id = 123;
```

BALANCE
2240

```
begin
  do_withdrawal('650707-1111',123,3000);
end;
/
```

```
ORA-20001: You do not have enough money in your account! ORA-06512: at "SQL_DKPWOAHFQVBTAZSMFCCQLGAF.BIFER_WITHDRAWAL", line 3
ORA-06512: at "SQL_DKPWOAHFQVBTAZSMFCCQLGAF.DO_WITHDRAWAL", line 10
ORA-06512: at line 2
ORA-06512: at "SYS.DBMS_SQL", line 1721
```

```
begin
    do_withdrawal('861124-4478',123,100);
end;
/
```

```
ORA-20001: Unauthorized user! ORA-06512: at "SQL_DKPWOAHFQVBTAZSMFCCQLGAF.DO_WITHDRAWAL", line 8
ORA-06512: at line 2
ORA-06512: at "SYS.DBMS_SQL", line 1721
```

Processing Transfers

We create a procedure called `do_transfer` that inserts a row into the **transfer** table. Like the withdrawal procedure, it declares an `unauthorized` exception, raised through `get_authority` when the customer lacks authority over the source account. If unauthorized, the transaction is stopped and *"Unauthorized user!"* is printed. If authorized, the procedure prints the balances of both accounts after the transaction.

```
create or replace procedure do_transfer(
    p_cust_id      in transfer.cust_id%type,
    p_from_acc_id  in transfer.from_acc_id%type,
    p_to_acc_id    in transfer.to_acc_id%type,
    p_amount       in transfer.amount%type)
as
begin
    if (get_authority(p_cust_id, p_from_acc_id)=0) then
        raise_application_error(-20001,'Unauthorized user!');
    else
        insert into transfer(cust_id, from_acc_id, to_acc_id, amount, date_time)
        values(p_cust_id, p_from_acc_id, p_to_acc_id, p_amount, sysdate);
        commit;
        dbms_output.put_line('The current balance of the account
        ||p_from_acc_id||' is '||get_balance(p_from_acc_id)||' sek. ');
        dbms_output.put_line('The current balance of the account
        ||p_to_acc_id ||' is '||get_balance(p_to_acc_id) ||' sek. ');
    end if;
end;
/
```

Testing the Transfer Procedure

Finally, we test `do_transfer` to move money between accounts and confirm that all the triggers behave as expected:

```
begin
    do_transfer('650707-1111',123,8896,1500);
end;
/
```

```
Statement processed.
The current balance of the account 123 is 740 sek.
The current balance of the account 8896 is 1500 sek.
```

```
select balance
from account
where acc_id in (123,8896);
```

BALANCE
740
1500

```
begin
do_transfer('861124-4478',123,8896,600);
end;
/
```

```
ORA-20001: Unauthorized user! ORA-06512: at "SQL_DKPHOAHFQVBTAZSMWFCCQLGAF.DO_TRANSFER", line 9
ORA-06512: at line 2
ORA-06512: at "SYS.DBMS_SQL", line 1721
```

```
begin
do_transfer('650707-1111',123,8896,2000);
end;
/
```

```
ORA-20001: You do not have enough money in your account! ORA-06512: at "SQL_DKPHOAHFQVBTAZSMWFCCQLGAF.BIFER_TRANSFER", line 3
ORA-06512: at "SQL_DKPHOAHFQVBTAZSMWFCCQLGAF.DO_TRANSFER", line 11
ORA-06512: at line 2
ORA-06512: at "SYS.DBMS_SQL", line 1721
```